

Assignment
RollNumber:202512026
Name:Santo Giriso David
computer programming c++

Here is a complete, modular C++ implementation for the Student Management System based on the requirements in your assignment sheet.

Because the assignment asks for a **Student Object** with constructors, getters, and setters, using a class instead of a traditional C-style struct is the standard approach in C++ OOP to encapsulate the data.

1. student.h (Header File)

This file defines the Student object blueprint and declares the global management functions that main.cpp will call.

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>
#include <vector>

[span_0](start_span)// Define the Student Object[span_0](end_span)
class Student {
private:
    std::string studentID;
    std::string name;
    int age;
    std::string course;

public:
    [span_1](start_span)// Constructors[span_1](end_span)
    Student();
    Student(std::string id, std::string name, int age, std::string course);

    [span_2](start_span)// Getters[span_2](end_span)
    std::string getStudentID() const;
    std::string getName() const;
    int getAge() const;
    std::string getCourse() const;

    [span_3](start_span)// Setters[span_3](end_span)
    void setStudentID(std::string id);
    void setName(std::string name);
    void setAge(int age);
    void setCourse(std::string course);
};
```

```
[span_4](start_span)// Function Declarations[span_4](end_span)
void addStudent();
void displayStudents();
void searchStudent();
void updateStudent();
void deleteStudent();

#endif
```

2. student.cpp (Source File)

This file implements the Student class methods and handles the logic for managing the list of students using a global std::vector.

```
#include "student.h"
#include <iostream>
#include <vector>
#include <iomanip>

// Global vector to store student records
std::vector<Student> studentList;

// Default Constructor
Student::Student() : studentID(""), name(""), age(0), course("") {}

// Parameterized Constructor
Student::Student(std::string id, std::string name, int age, std::string course) {
    this->studentID = id;
    this->name = name;
    this->age = age;
    this->course = course;
}

// Getters Implementation
std::string Student::getStudentID() const { return studentID; }
std::string Student::getName() const { return name; }
int Student::getAge() const { return age; }
std::string Student::getCourse() const { return course; }

// Setters Implementation
void Student::setStudentID(std::string id) { studentID = id; }
void Student::setName(std::string n) { name = n; }
void Student::setAge(int a) { age = a; }
void Student::setCourse(std::string c) { course = c; }

[span_6](start_span)[span_7](start_span)// Function to Add a
Student[span_6](end_span)[span_7](end_span)
```

```

void addStudent() {
    std::string id, name, course;
    int age;

    std::cout << "\n--- Add New Student ---\n";
    std::cout << "Enter Student ID: ";
    std::cin >> id;
    std::cin.ignore(); // Clear buffer

    // Check if ID already exists
    for (const auto& s : studentList) {
        if (s.getStudentID() == id) {
            std::cout << "Error: A student with this ID already exists!\n";
            return;
        }
    }

    std::cout << "Enter Name: ";
    std::getline(std::cin, name);
    std::cout << "Enter Age: ";
    std::cin >> age;
    std::cin.ignore();
    std::cout << "Enter Course: ";
    std::getline(std::cin, course);

    // Create and push the object into the vector
    Student newStudent(id, name, age, course);
    studentList.push_back(newStudent);
    std::cout << "Student added successfully!\n";
}

[span_8](start_span)[span_9](start_span)// Function to Display All
Students[span_8](end_span)[span_9](end_span)
void displayStudents() {
    if (studentList.empty()) {
        std::cout << "\nNo student records found.\n";
        return;
    }

    std::cout << "\n-----\n";
    std::cout << std::left << std::setw(15) << "ID"
        << std::setw(20) << "Name"
        << std::setw(10) << "Age"
        << std::setw(15) << "Course" << "\n";
    std::cout << "-----\n";

    for (const auto& s : studentList) {
        std::cout << std::left << std::setw(15) << s.getStudentID()

```

```

        << std::setw(20) << s.getName()
        << std::setw(10) << s.getAge()
        << std::setw(15) << s.getCourse() << "\n";
    }
    std::cout << "-----\n";
}

```

[span_10](start_span)[span_11](start_span)// Function to Search for a Student by ID[span_10](end_span)[span_11](end_span)

```

void searchStudent() {
    if (studentList.empty()) {
        std::cout << "\nNo records available to search.\n";
        return;
    }

    std::string id;
    std::cout << "\nEnter Student ID to Search: ";
    std::cin >> id;

    for (const auto& s : studentList) {
        if (s.getStudentID() == id) {
            std::cout << "\nStudent Found:\n";
            std::cout << "ID: " << s.getStudentID() << "\n";
            std::cout << "Name: " << s.getName() << "\n";
            std::cout << "Age: " << s.getAge() << "\n";
            std::cout << "Course: " << s.getCourse() << "\n";
            return;
        }
    }
    std::cout << "Student with ID " << id << " not found.\n";
}

```

[span_12](start_span)[span_13](start_span)// Function to Update Student Information[span_12](end_span)[span_13](end_span)

```

void updateStudent() {
    if (studentList.empty()) {
        std::cout << "\nNo records available to update.\n";
        return;
    }

    std::string id;
    std::cout << "\nEnter Student ID to Update: ";
    std::cin >> id;

    for (auto& s : studentList) {
        if (s.getStudentID() == id) {
            std::string newName, newCourse;
            int newAge;

```

```

        std::cout << "\nStudent Found! Enter New Details:\n";
        std::cin.ignore();
        std::cout << "Enter New Name: ";
        std::getline(std::cin, newName);
        std::cout << "Enter New Age: ";
        std::cin >> newAge;
        std::cin.ignore();
        std::cout << "Enter New Course: ";
        std::getline(std::cin, newCourse);

        s.setName(newName);
        s.setAge(newAge);
        s.setCourse(newCourse);

        std::cout << "Student record updated successfully!\n";
        return;
    }
}
std::cout << "Student with ID " << id << " not found.\n";
}

[span_14](start_span)[span_15](start_span)// Function to Delete a
Student[span_14](end_span)[span_15](end_span)
void deleteStudent() {
    if (studentList.empty()) {
        std::cout << "\nNo records available to delete.\n";
        return;
    }

    std::string id;
    std::cout << "\nEnter Student ID to Delete: ";
    std::cin >> id;

    for (auto it = studentList.begin(); it != studentList.end(); ++it) {
        if (it->getStudentID() == id) {
            studentList.erase(it);
            std::cout << "Student record deleted successfully!\n";
            return;
        }
    }
    std::cout << "Student with ID " << id << " not found.\n";
}

```

3. main.cpp (Main Program File)

This houses the interactive menu loop allowing users to direct the application flow until they select exit.

```
#include <iostream>
#include "student.h"
```

```
int main() {
    int choice;
```

```
[span_17](start_span)// Menu loop[span_17](end_span)
```

```
do {
    std::cout << "\n=====\\n";
    std::cout << "  STUDENT MANAGEMENT SYSTEM  \\n";
    std::cout << "=====\\n";
    std::cout << "1. Add Student\\n";
    std::cout << "2. Display All Students\\n";
    std::cout << "3. Search Student by ID\\n";
    std::cout << "4. Update Student Information\\n";
    std::cout << "5. Delete Student\\n";
    std::cout << "6. Exit\\n";
    std::cout << "=====\\n";
    std::cout << "Enter your choice (1-6): ";
    std::cin >> choice;
```

```
[span_18](start_span)// Route choice to appropriate function[span_18](end_span)
```

```
switch (choice) {
    case 1:
        addStudent();
        break;
    case 2:
        displayStudents();
        break;
    case 3:
        searchStudent();
        break;
    case 4:
        updateStudent();
        break;
    case 5:
        deleteStudent();
        break;
    case 6:
        std::cout << "\\nExiting the program. Goodbye!\\n";
        break;
    default:
        std::cout << "\\nInvalid choice! Please select a valid option (1-6).\\n";
}
```

```
} while (choice != 6); [span_19](start_span)// End only when Exit is
chosen[span_19](end_span)
```

```
    return 0;  
}
```

Final Deadline Reminders

- **GitHub Submission:** Push these 3 files (student.h, student.cpp, main.cpp) to a repository and email the link to **kamatekatende@gmail.com** before **02:30 PM today**.
- **Hardcopy:** Do not forget to print out your source code files or bring them as requested for the **8:30 AM** review session.
- **Group Details:** Make sure your document cover sheet lists up to **5 students** along with everyone's **roll numbers**.